indexes online without stopping PostgreSQL, which is critical for a running application.

For the Performance database, it's being refreshed offline. The REINDEX command can be run without CONCURRENTLY because there are no other concurrent users of the indexes. Without that option the REINDEX operation will run faster.

Although it may not be strictly necessary, practice rebuilding the index index_users_on_email for the users table by running this statement from psql:

```
REINDEX INDEX CONCURRENTLY index_users_on_email;
```

You've now performed VACUUMM, ANALYZE, and REINDEX operations following a big update.

With those maintenance operations, the users table is optimized again for use by the application.

Performing a single UPDATE statement on a large table with hundreds of thousands of rows may not be feasible. Instead you will likely want to break up the updates into batches. How might you do that?

Performing Modifications In Batches

You've written *Functions* in SQL and learned they can be written with the PL/pgSQL Procedural Language.

PostgreSQL offers another capability that extends functions further, called *Procedures*, introduced in version 11. You'll use a Procedure to perform batched updates, calling a function within the Procedure implementation body.

Procedures can be used to provide control flow constructs like looping. As a Ruby programmer, you've written a lot of loops before, but you may not have written loops in PostgreSQL code.

Below you'll find a Procedure that takes the Direct Updates UPDATE statement approach and breaks it up into batches.

The Primary Key id column is used to select with for a range of values. The variable for controlling the batch size can act as a configuration point depending on whether your server is capable of more updates at a time or fewer. The default size is 1000. You may wish to increase this to 10000 or more if you have a big server.